Prompt Tecnico:

LENGUAJE DE PROGRAMACIÓN:

- Python 3.x
- Utilizar exclusivamente la biblioteca estándar `threading` para la concurrencia (sin frameworks externos de terceros)

PLATAFORMA:

- Aplicación de consola (CLI), multihilo, ejecutable en cualquier sistema operativo con Python 3.10 o superior
- Sin interfaz gráfica; toda la interacción y visualización del estado del sistema debe hacerse por salida estándar, de forma clara y ordenada (fácil de seguir mientras corre)

FUNCIONALIDAD A IMPLEMENTAR:

- Simular un sistema de procesamiento de pedidos de un e-commerce mediante el patrón Productor-Consumidor, con múltiples hilos productores y múltiples hilos consumidores compartiendo un buffer acotado (cola de capacidad máxima fija)
- Soportar dos niveles de prioridad de pedidos (urgente / normal), de modo que los urgentes se procesen antes, sin que los normales sufran inanición indefinida
- Generar un resumen final con métricas de throughput y tiempos de espera al terminar la simulación

REQUISITOS ESPECÍFICOS:

- Implementar N hilos productores (N configurable, mínimo 3) que generen pedidos a intervalos aleatorios (entre 100 y 500 ms), cada uno con: id único, prioridad (urgente/normal, con probabilidad configurable, ej. 20% urgente) y marca de tiempo de creación
- Implementar M hilos consumidores (M configurable, mínimo 2) que tomen pedidos del buffer y simulen su procesamiento con una demora aleatoria (entre 200 y 800 ms)
- El buffer compartido debe tener capacidad máxima fija (ej. 10 pedidos); productores y consumidores deben bloquearse sin espera activa cuando el buffer esté lleno o vacío, respectivamente
- Los pedidos urgentes deben tener prioridad de extracción, pero el diseño debe garantizar que un pedido normal nunca espere de forma indefinida (por ejemplo, envejeciendo su prioridad tras cierto tiempo de espera)
- Toda la sincronización debe implementarse con primitivas de `threading` (Lock, Condition, Semaphore o similares); no se permite ninguna forma de polling/espera activa para coordinar productores y consumidores
- El programa debe finalizar de forma ordenada (graceful shutdown) tras procesar una cantidad total configurable de pedidos (ej. 50), usando el patrón "poison pill" o un mecanismo equivalente, sin dejar hilos colgados ni recursos sin liberar

- Debe registrarse (logging thread-safe) cada evento relevante — creación, encolado, desencolado e inicio/fin de procesamiento — con timestamp, id de hilo e id de pedido
- Al finalizar, imprimir un resumen con: pedidos procesados por prioridad, tiempo de espera promedio en cola por prioridad, y tiempo total de ejecución
- El código debe organizarse en funciones/clases con nombres descriptivos y comentarios que justifiquen cada decisión de sincronización

RESTRICCIONES:

- No usar `multiprocessing` ni `asyncio`: la concurrencia debe resolverse exclusivamente con `threading`, para poder analizar explícitamente los mecanismos de sincronización que pide la rúbrica
- Si se usa `queue.PriorityQueue`, debe complementarse con lógica explícita de envejecimiento de prioridad — no alcanza con delegarle la prioridad "mágicamente" sin justificar cómo se evita la inanición
- El tiempo total de ejecución de la simulación no debe superar los 60 segundos
- No debe haber variables globales mutables compartidas sin protección: todo estado compartido entre hilos tiene que estar resguardado por el mecanismo de sincronización correspondiente
- No se permite depender de una interrupción manual (Ctrl+C) como mecanismo de finalización: el programa debe terminar solo, de forma controlada